



Verification and Validation Report: RoCam

Team #3, SpaceY

Zifan Si

Jianqing Liu

Mike Chen

Xiaotian Lou

March 9, 2026

1 Revision History

Date	Version	Notes
2026-03-04	1.0	Add Document Evaluation
2026-03-05	1.1	Add Unit Tests based on run
2026-03-09	1.2	Add Reflection Section and small fixes
2026-04-05	1.3	Rev 1: Addressed TA and peer feedback; improved Pass text accessibility; clarified lab vs field testing scope; added planned future improvements; reduced duplication with VnV Plan; verified traceability

2 Symbols, Abbreviations, and Acronyms

Symbol	Description
T	Test
YOLO	You Only Look Once – an existing object detection model
CNN	Convolutional Neural Network
Jetson	Abbreviation for Jetson Nano Orin, the proposed board for this project
STM32	Abbreviation for STM32 microcontroller, the proposed microcontroller for this project

2.1 Unit Test ID Dictionary

The table below defines the test ID commands used throughout this report and serves as a quick dictionary for repeated references. Clicking any `\tid{test-id}` reference jumps to the corresponding entry here.

Table 1: Unit Test ID Dictionary

Test ID	Dictionary Entry
test-acquire-frame	acquire frame
test-api-client-automatic-discovery	api client automatic discovery
test-api-client-methods	api client methods
test-api-client-urls	api client urls
test-api-endpoints-exist	api endpoints exist
test-api-error-responses	api error responses
test-api-recordings-crud	api recordings crud
test-api-validation	api validation
test-app-routing	app routing
test-bilingual-ui	bilingual ui
test-bounding-box-center	bounding box center

Continued on next page

Table 1 – Continued from previous page

Test ID	Dictionary Entry
test-bounding-box-collection	bounding box collection
test-bounding-box-rotate-90	bounding box rotate 90
test-color-vision-deficiency	color vision deficiency
test-configuration-menu	configuration menu
test-confirmation-prompts	confirmation prompts
test-consistent-units	consistent units
test-control-page-bbox-overlay	control page bbox overlay
test-control-page-drag	control page drag
test-control-page-render	control page render
test-control-process-main-entry	control process main entry
test-controls-card-render	controls card render
test-cv-format-time	cv format time
test-cv-management-exception-safety	cv management exception safety
test-cv-management-osd-send	cv management osd send
test-cv-management-pause-resume	cv management pause resume
test-cv-management-recording-commands	cv management recording commands
test-cv-management-recv-loop	cv management recv loop
test-cv-update-osd	cv update osd
test-database-allocate	database allocate
test-database-error-handling	database error handling
test-database-list-recordings	database list recordings

Continued on next page

Table 1 – Continued from previous page

Test ID	Dictionary Entry
test-database-path-traversal-protection	database path traversal protection
test-database-rename-delete	database rename delete
test-database-space-usage	database space usage
test-dataclass-construction	dataclass construction
test-disconnect-camera	disconnect camera
test-disconnect-hdmi	disconnect hdmi
test-disk-full	disk full
test-exit-gimbal-range	exit gimbal range
test-formatters-bytes	formatters bytes
test-formatters-degrees	formatters degrees
test-formatters-duration	formatters duration
test-formatters-edge-cases	formatters edge cases
test-formatters-internationalization	formatters internationalization
test-formatters-temperature	formatters temperature
test-gimbal-angle-commands	gimbal angle commands
test-gimbal-crc8	gimbal crc8
test-gimbal-info-query	gimbal info query
test-gimbal-interface	gimbal interface
test-gimbal-led-control	gimbal led control
test-gimbal-packet-construction	gimbal packet construction
test-hdmi-output	hdmi output

Continued on next page

Table 1 – Continued from previous page

Test ID	Dictionary Entry
test-i18n-activation	i18n activation
test-immediate-ui-feedback	immediate ui feedback
test-ip4-addresses	ip4 addresses
test-ipc-buffer-memory-safety	ipc buffer memory safety
test-ipc-buffer-overflow-handling	ipc buffer overflow handling
test-ipc-buffer-receiver	ipc buffer receiver
test-ipc-buffer-sender	ipc buffer sender
test-language-selector	language selector
test-livestream-construction	livestream construction
test-livestream-process-loop	livestream process loop
test-log-operations	log operations
test-main-dispatch-control	main dispatch control
test-main-dispatch-cv	main dispatch cv
test-main-dispatch-livestream	main dispatch livestream
test-main-dispatch-transcode	main dispatch transcode
test-main-dispatch-unknown	main dispatch unknown
test-manage-recordings	manage recordings
test-manual-arm-mode	manual arm mode
test-manual-control	manual control
test-manual-idle-mode	manual idle mode
test-navbar	navbar

Continued on next page

Table 1 – Continued from previous page

Test ID	Dictionary Entry
test-network-disconnect	network disconnect
test-no-collection-pii	no collection pii
test-preview-real-time	preview real time
test-recording	recording
test-recordings-delete	recordings delete
test-recordings-page-empty	recordings page empty
test-recordings-page-list	recordings page list
test-recordings-page-loading	recordings page loading
test-recordings-rename	recordings rename
test-report-errors	report errors
test-return-to-idle	return to idle
test-rocam-provider-error-handling	rocam provider error handling
test-rocam-provider-initialization	rocam provider initialization
test-rocam-provider-polling	rocam provider polling
test-scheduler-validation	scheduler validation
test-sse-broadcast-performance	sse broadcast performance
test-sse-thread-safety	sse thread safety
test-state-error-handling	state error handling
test-state-management-arm-disarm	state management arm disarm
test-state-management-download-pause	state management download pause
test-state-management-manual-move	state management manual move

Continued on next page

Table 1 – Continued from previous page

Test ID	Dictionary Entry
test-state-management-manual-move-clamped	state management manual move clamped
test-state-management-recording	state management recording
test-state-management-recording-only-noops	state management recording only noops
test-state-management-recording-only-status	state management recording only status
test-state-status-performance	state status performance
test-stop-immediately	stop immediately
test-system-status-caching	system status caching
test-system-status-callback-updates	system status callback updates
test-system-status-card	system status card
test-tracking	tracking
test-tracking-angle-clamping	tracking angle clamping
test-tracking-queue-behavior	tracking queue behavior
test-tracking-stop	tracking stop
test-tracking-worker-control-law	tracking worker control law
test-transcode-bus-call	transcode bus call
test-transcode-pipe-cleanup	transcode pipe cleanup
test-transcode-read-log	transcode read log
test-transcode-routes-404	transcode routes 404
test-transcode-shader-probe	transcode shader probe
test-transcode-streaming	transcode streaming
test-transition-to-tracking	transition to tracking

Continued on next page

Table 1 – Continued from previous page

Test ID	Dictionary Entry
test-ui-error-handling	ui error handling
test-ui-loading-states	ui loading states
test-use-rocam-hook	use rocam hook
test-validate-commands	validate commands
test-visual-indicators	visual indicators
test-watchdog-clear	watchdog clear
test-watchdog-refresh	watchdog refresh
test-watchdog-timeout-accuracy	watchdog timeout accuracy

Contents

1	Revision History	i
2	Symbols, Abbreviations, and Acronyms	ii
2.1	Unit Test ID Dictionary	ii
3	Document Evaluation	1
3.1	SRS Evaluation	1
3.1.1	Abstraction	1
3.1.2	Testability	1
3.1.3	Ambiguity	2
3.1.4	Completeness	2
3.1.5	Spelling and Grammar	3
3.1.6	Ease of Understanding	3
3.2	Design Evaluation	3
3.2.1	Consistency With Requirements	3
3.2.2	Modularity and Interfaces	3
3.2.3	Maintainability	4
3.3	Verification and Validation Plan Evaluation	4
3.3.1	Completeness	4
3.3.2	Unambiguous	4
3.3.3	Feasibility	4
3.3.4	Overall Objective Accomplishment	4
4	Functional Requirements Evaluation	5
4.0.1	System Tests	5
5	Nonfunctional Requirements Evaluation	8
5.1	Usability	8
5.2	Performance	11
5.3	Robustness and Reliability	12
6	Comparison to Existing Implementation	14
7	Unit Testing	14
7.1	Backend Unit Tests (<code>pytest</code>)	14
7.1.1	Module: State Management	14
7.1.2	Module: API Gateway	16
7.1.3	Module: Recording Database	17
7.1.4	Module: Gimbal Abstraction	18
7.1.5	Module: Tracking	19
7.1.6	Module: CV Process Management	19
7.1.7	Module: Live Video Process Management	20
7.1.8	Module: Transcode API	21
7.1.9	Module: Transcode Process	21

7.1.10	Module: CV Process	22
7.1.11	Module: Common IPC	22
7.1.12	Module: Common IPC Buffer	23
7.1.13	Module: Common Watchdog	23
7.1.14	Module: System Status	24
7.1.15	Module: Common Utils	24
7.1.16	Module: Recording-Only State Management	25
7.1.17	Module: Main Entry Point	25
7.1.18	Module: SSE	26
7.2	Frontend Unit Tests (Vitest)	27
8	Changes Due to Testing	32
8.1	Planned Future Improvements	33
9	Automated Testing	34
10	Trace to Requirements	35
11	Trace to Modules	35
12	Code Coverage Metrics	38

List of Tables

1	Unit Test ID Dictionary	ii
2	Usability and Safety Changes from V&V	32
3	UI and Recording Changes from V&V	33
4	Backend Validation Changes from V&V	33
5	Planned Future Improvements Identified During V&V	34
6	Traceability Between Unit Test Cases and Backend Modules	36
7	Traceability Between Unit Test Cases and Frontend Modules	37
8	Code Coverage Metrics	38

List of Figures

This document reports on the verification and validation activities for RoCam, evaluating the [SRS Vision-Guided Tracker Team \(2025c\)](#), design documents ([MG Vision-Guided Tracker Team \(2025a\)](#), [MIS Vision-Guided Tracker Team \(2025b\)](#)), and [VnV Plan Vision-Guided Tracker Team \(2025d\)](#), and presenting the results of unit tests and system tests.

3 Document Evaluation

3.1 SRS Evaluation

3.1.1 Abstraction

The [SRS Vision-Guided Tracker Team \(2025c\)](#) was reviewed to ensure requirements describe *what* RoCam must do from a stakeholder/operator perspective (camera operator, rocketry team) rather than *how* it is implemented (specific CV models, firmware details, or UI framework choices). Overall, the Functional Requirements (FRs) are written at a suitable level of abstraction: they describe externally observable behaviours such as acquiring frames, displaying a live preview, allowing manual control, tracking/arming transitions, and recording management.

The Non-Functional Requirements (NFRs) were also reviewed and are generally abstract (e.g., usability, performance, safety, reliability). Where an NFR implicitly assumes a particular technical solution (e.g., a specific fps target or a specific latency measurement method), this is acceptable if the [SRS Vision-Guided Tracker Team \(2025c\)](#) also defines a clear, measurable fit criterion.

3.1.2 Testability

The FRs are mostly testable because they describe observable outcomes that can be checked via logs, UI state, or recorded artifacts (e.g., a created video file). However, due to the final project scope constraint that **only field testing is out of scope** (unit and system tests are executed), some FRs are not directly testable end-to-end within this iteration. *Note on hardware-in-the-loop system tests:* System tests documented in this report (e.g., camera acquisition, gimbal control, tracking) were conducted in a **controlled lab environment** using real hardware (Jetson Orin Nano, camera, gimbal, and a hand-held model rocket). This is distinct from **field testing**—validation at an actual rocket launch site with live flights—which remains out of scope due to scheduling and logistical constraints. To preserve testability under the reduced scope, FR evaluation in this project is performed through:

- Unit tests for isolated logic (e.g., state transitions, validation, file naming/path logic, error formatting),
- Component-level tests where a module boundary exists (e.g., UI state reducers, API request formatting, parsing),
- Stakeholder review feedback for user-facing correctness/priority (without live flight validation).

For the NFRs, testability varies:

- **Performance:** testable if the [SRS Vision-Guided Tracker Team \(2025c\)](#) states measurable thresholds (e.g., 1080p60 I/O, latency targets), but in this project we can only partially test performance via unit/component tests and profiling of specific pipelines rather than full system measurement.
- **Usability:** testable through walkthrough protocols and checklists (screenshots, task completion observations), but results are less precise than instrumentation-based studies.
- **Safety:** testable for software guards (confirmation prompts, manual arming only, safe idle transitions) via unit tests and static inspection; physical safety behaviours at the gimbal/hardware layer are limited by out-of-scope hardware validation.

3.1.3 Ambiguity

The [SRS Vision-Guided Tracker Team \(2025c\)](#) wording is largely clear, but several items can be improved to reduce interpretation differences during verification:

- **Terms that need operational definitions:** phrases such as “real-time”, “smooth”, “fluid”, “clear error message”, and “maintains lock” should map to concrete thresholds (fps, latency, allowed dropped frames, maximum time-to-reacquire, required UI elements).
- **State-related FRs:** FRs related to *armed*, *tracking*, and *idle* benefit from explicitly defining entry/exit conditions and time bounds (e.g., “within 2 s of sustained loss, transition to idle”).
- **Recording management:** where FRs mention “download” or “list”, the expected metadata should be explicit (name format, timestamp source, duration computation, sorting rule).

These ambiguities do not block implementation but do increase the risk of disagreement during V&V. Where possible, they are handled in this project by encoding assumptions as unit-testable acceptance criteria.

3.1.4 Completeness

For completeness, we reviewed whether the [SRS Vision-Guided Tracker Team \(2025c\)](#) covers the primary stakeholder goals for RoCam: acquisition, operator control, automated tracking behaviour, recording workflow, and safe operation. At a high level, the [SRS Vision-Guided Tracker Team \(2025c\)](#) is close to complete for Rev0 scope. The main completeness concerns arise from:

- **Out-of-scope verification constraints:** since field tests are not executed in this iteration, any FR/NFR that requires integrated hardware measurement (true end-to-end latency, outdoor robustness, real launch tracking) cannot be fully verified. The [SRS Vision-Guided Tracker Team \(2025c\)](#) should explicitly mark such requirements as requiring future work or later verification stages.

- **Third-party dependencies:** camera/gimbal behaviour is assumed per manufacturer documentation; the [SRS Vision-Guided Tracker Team \(2025c\)](#) should clarify which behaviours are *assumed* vs *verified by this team*.
- **Safety and hazard linkage:** any hazard mitigations (e.g., emergency stop, confirmation prompts, manual arm gating) should be explicitly referenced in either FRs or NFRs so that traceability from the Hazard Analysis to requirements is direct.

3.1.5 Spelling and Grammar

A pass over the [SRS Vision-Guided Tracker Team \(2025c\)](#) found no major grammar defects; remaining issues are primarily minor consistency fixes (capitalization, hyphenation, consistent use of abbreviations such as “UI”, “fps”, “gimbal”). In the final report, we standardize terminology across documents ([SRS Vision-Guided Tracker Team \(2025c\)](#), [MG Vision-Guided Tracker Team \(2025a\)](#), [MIS Vision-Guided Tracker Team \(2025b\)](#), [VnV Plan Vision-Guided Tracker Team \(2025d\)](#), and this report) to reduce confusion during traceability review.

3.1.6 Ease of Understanding

The [SRS Vision-Guided Tracker Team \(2025c\)](#) is readable and structured, but ease-of-understanding can be improved by:

- Making requirement IDs clickable links in the PDF (or ensuring the traceability tables include enough context so readers do not need to search manually).
- Adding brief legends/notes to diagrams (if any) explaining symbols and data flow, especially when multiple modules interact (Jetson pipeline, STM32 controller, UI).
- Including a short glossary of user-facing terms (armed/tracking/idle, “preview”, “overlay”, “recording”) so interpretation is consistent.

3.2 Design Evaluation

3.2.1 Consistency With Requirements

We reviewed the [MG Vision-Guided Tracker Team \(2025a\)](#)/[MIS Vision-Guided Tracker Team \(2025b\)](#) against the [SRS Vision-Guided Tracker Team \(2025c\)](#) to ensure that each major requirement has a clear design owner (vision pipeline, control firmware/interface, UI/recording). Overall, the decomposition is aligned with RoCam’s core functions. The main design-to-requirement risk is where requirements imply integrated behaviour (e.g., tracking quality under occlusion) but the design only specifies unit-level behaviour without an executable system test under the current scope.

3.2.2 Modularity and Interfaces

The design is modular across the main subsystems (vision, control, UI). Interfaces should remain stable and well-documented (commands, telemetry, timing, and error reporting). This modularity supports unit testing under the constrained scope by allowing logic verification at module boundaries even when field tests are not executed.

3.2.3 Maintainability

Maintainability is supported by separation of concerns and internal review via pull requests. The main maintainability improvement opportunity is to enforce consistent error/telemetry schemas between backend and frontend so tests and logs remain stable as features evolve.

3.3 Verification and Validation Plan Evaluation

3.3.1 Completeness

The [VnV Plan Vision-Guided Tracker Team \(2025d\)](#) contains detailed system test descriptions. Unit testing and system testing are executed; only field testing is out of scope. Therefore, completeness in the plan is interpreted as:

- Unit tests and system tests executed,
- Executed evidence coming from unit tests, system tests, and review artifacts.

To avoid mismatch with the report, the report will clearly label which planned tests were executed and which were not, and will provide unit-test evidence that covers the same FR intent where feasible.

3.3.2 Unambiguous

The [VnV Plan Vision-Guided Tracker Team \(2025d\)](#) is mostly unambiguous, especially where checklists specify concrete actions. Ambiguity exists mainly where tests describe qualitative outcomes (e.g., “smooth”, “clear error message”) without explicit acceptance criteria. Where field tests are not executed, ambiguity is reduced by replacing them in the report with unit-testable criteria or explicit “not executed” markings.

3.3.3 Feasibility

The [VnV Plan Vision-Guided Tracker Team \(2025d\)](#) is feasible under the revised scope because it limits execution to unit tests, system tests, and stakeholder review. Only field testing (live rocket launches, hardware field conditions, external reviewers) is explicitly excluded. This prevents over-claiming and keeps V&V achievable within available time and access.

3.3.4 Overall Objective Accomplishment

Under the revised scope, the [VnV Plan Vision-Guided Tracker Team \(2025d\)](#) still supports the core objectives:

- **Document confidence:** internal checklist-based review and traceability tables improve confidence in [SRS Vision-Guided Tracker Team \(2025c\)](#)/design consistency.
- **Nonfunctional focus:** selected NFRs (e.g., usability walkthrough, performance characterization at module level) can be partially supported with evidence, with limitations clearly stated.
- **Implementation confidence:** unit tests and system tests provide concrete, repeatable evidence that critical logic behaves as expected.

4 Functional Requirements Evaluation

Unit testing and system testing are executed; only field testing is out of scope. Functional requirement verification is achieved through unit tests (Section 7) and the following system tests. All system tests **passed**.

4.0.1 System Tests

Note: Full test descriptions, rationale, and acceptance criteria for each system test are documented in the [VnV Plan Vision-Guided Tracker Team \(2025d\)](#). This section reports **execution results only** to avoid unnecessary duplication between the two documents.

Area: Video Acquisition and Output

Test ID	test-acquire-frame
Initial State	System powered off; camera connected; HDMI monitor attached
Input	Power on the system and allow it to fully initialize
Expected Results	Within 10s of boot, logs show that the camera is acquiring frames
Success	Pass

Test ID	test-disconnect-camera
Initial State	System initialized; preview and HDMI output active
Input	Physically disconnect the camera from the system
Expected Results	UI and HDMI output show a clear error message; no crash
Success	Pass

Test ID	test-disconnect-hdmi
Initial State	System initialized; preview and HDMI output active
Input	Physically disconnect the HDMI cable
Expected Results	UI shows a clear error message; no crash
Success	Pass

Test ID	test-hdmi-output
Initial State	System initialized; idle state; preview and HDMI output active
Input	Move the gimbal manually through the UI
Expected Results	HDMI shows 1080p60 camera feed with text overlay of pan/tilt angles
Success	Pass

Test ID	test-preview-real-time
Initial State	System initialized; idle state; preview and HDMI output active
Input	Move the gimbal manually through the UI
Expected Results	Preview updates fluidly (≥ 15 fps); displays current pan/tilt angles
Success	Pass

Test ID	test-network-disconnect
Initial State	System initialized; preview and HDMI output active
Input	Disconnect the system from the network for 30s; reconnect
Expected Results	System keeps functioning; UI shows clear error; session restores within 5s on reconnect
Success	Pass

Area: Recording

Test ID	test-recording
Initial State	System initialized; idle state; sufficient disk space
Input	Press “Start Recording”, wait 10s, then press “Stop Recording”
Expected Results	A 10 second 1080p60 video file is created and saved to disk
Success	Pass

Test ID	test-disk-full
Initial State	System initialized; idle state; storage nearly full
Input	Start recording and allow disk to fill
Expected Results	UI shows “Storage Full” alert; recording stopped and file saved; no crash
Success	Pass

Test ID	test-manage-recordings
Initial State	At least two prior recordings exist
Input	Open recordings list; download latest and older file; delete a test clip
Expected Results	List shows correct entries; downloads succeed; delete removes selected item only
Success	Pass

Area: Tracking

Test ID	test-manual-control
Initial State	System initialized; idle state; preview and HDMI output active
Input	Issue manual pan and tilt commands via UI
Expected Results	Gimbal follows commands smoothly
Success	Pass

Test ID	test-transition-to-tracking
Initial State	System initialized; idle state; preview and HDMI output active
Input	Arm the system via UI; wave a model rocket in front of the camera
Expected Results	System transitions to tracking when rocket detected; UI shows tracking state
Success	Pass

Test ID	test-tracking
Initial State	System initialized; tracking state; preview and HDMI output active
Input	Continue waving the model rocket in front of the camera
Expected Results	Gimbal continuously adjusts to keep rocket in frame
Success	Pass

Test ID	test-return-to-idle
Initial State	System initialized; tracking state; preview and HDMI output active
Input	Occlude the model rocket
Expected Results	Within 2s of sustained loss, system transitions to idle; UI indicates “target lost”
Success	Pass

Test ID	test-exit-gimbal-range
Initial State	System initialized; tracking state; preview and HDMI output active
Input	Move the model rocket outside of the gimbal’s range of motion
Expected Results	Gimbal moves to mechanical limit and stops; no errors; system remains responsive
Success	Pass

FR verification: Functional requirements are verified through unit tests (Section 7) and the system tests above. Unit tests cover state management, API endpoints, recording database, gimbal control, tracking logic, CV process management, and UI components.

5 Nonfunctional Requirements Evaluation

Nonfunctional requirements are verified through unit tests, system tests, and manual walkthrough tests (executed). Only field testing is out of scope. The following presents tests from the [VnV Plan Vision-Guided Tracker Team \(2025d\)](#) using the same table format as the Unit Testing section.

5.1 Usability

Unit tests and manual walkthrough tests verify usability-related behaviour.

Unit Tests

Test ID	test-ui-loading-states
Initial State	Components with loading/null states
Input	Null or undefined data
Expected Results	Loading spinners or placeholder content; no crashes
Success	Pass

Test ID	test-language-selector
Initial State	LanguageSelector component
Input	Language selection change
Expected Results	Language atom updated; i18n activated
Success	Pass

Test ID	test-configuration-menu
Initial State	ConfigurationMenu component
Input	Settings changes (temperature unit, drag sensitivity, invert drag)
Expected Results	Setting atoms updated with new values
Success	Pass

Manual Walkthrough Tests

Test ID	test-consistent-units
Initial State	System initialized; idle state; preview and HDMI output active
Input	Go through the UI twice, once with metric units, once with imperial units
Expected Results	Units are consistent and changeable
Success	Pass

Test ID	test-bilingual-ui
Initial State	System initialized; language = English
Input	Switch UI language to French and back; navigate all UI sections
Expected Results	All visible strings appear in selected language; no truncation; formats localize correctly
Success	Pass

Test ID	test-immediate-ui-feedback
Initial State	System idle; UI ready
Input	Trigger actions: button press, toggle, menu selection, recording, arming
Expected Results	Visible confirmation within 0.2s for each action
Success	Pass

Test ID	test-visual-indicators
Initial State	System initialized; speakers muted or disconnected
Input	Trigger statuses/alerts: armed, tracking, target lost, storage low/full, network disconnect, device error
Expected Results	Every status/alert presented visually; no action-critical info is audio-only
Success	Pass

Test ID	test-confirmation-prompts
Initial State	System idle; gimbal powered
Input	Attempt potentially hazardous actions: force home while tracking, hard stop, high-speed slew, firmware update during armed state, factory reset
Expected Results	Blocking confirmation dialog for each action; default is “Cancel”; action proceeds only after explicit confirmation
Success	Pass

Test ID	test-manual-idle-mode
Initial State	System armed or tracking
Input	Press “Idle” control while tracking
Expected Results	System transitions to idle within 1 s; autonomous commands cease; manual controls available
Success	Pass

Test ID	test-no-collection-pii
Initial State	System initialized; default configuration
Input	Inspect all settings, logs, exported files, filenames/paths, and network-configurable fields
Expected Results	No fields request or store personal identifiers; logs contain only technical data
Success	Pass

Test ID	test-color-vision-deficiency
Initial State	System initialized; all status indicators visible
Input	Review all status/alert states under simulated Protanopia/Deutanopia/Tritanopia
Expected Results	Each state distinguishable without relying on hue alone; contrast ratio $\geq 4.5:1$
Success	Pass

Code and Documentation Walkthrough

Test ID	test-manual-arm-mode
Initial State	N/A
Input	Inspect all state transition code and configuration defaults
Expected Results	System only enters armed mode when explicitly commanded by user, never automatically
Success	Pass

Test ID	test-gimbal-interface
Initial State	N/A
Input	Review interface specification with client: command set, telemetry, timing, safety notes
Expected Results	Client confirms interface documentation is complete, clear, and suitable
Success	Pass

Error Handling and Logging

Test ID	test-report-errors
Initial State	System initialized and idle
Input	Manually trigger hardware, communication, and software faults
Expected Results	Each error generates a visible and descriptive message on the UI
Success	Pass

Test ID	test-stop-immediately
Initial State	System operating in tracking mode
Input	Introduce fatal condition (gimbal communication loss or corrupted control process)
Expected Results	System immediately stops all automated operations and transitions to safe idle state
Success	Pass

Test ID	test-validate-commands
Initial State	System idle
Input	Enter invalid or out-of-range user commands through UI or API
Expected Results	System rejects invalid commands and displays validation error without sending to gimbal
Success	Pass

Test ID	test-log-operations
Initial State	System initialized
Input	Perform user operations: arming, tracking, recording
Expected Results	Each action logged with timestamp and operation details in system log
Success	Pass

5.2 Performance

Unit tests verify performance-related behaviour (status caching, SSE broadcast, watchdog timing).

Test ID	test-state-status-performance
Initial State	StateManagement with mocked dependencies and cached metrics
Input	status() called repeatedly
Expected Results	Status returned within milliseconds; cached values without recalculation
Success	Pass

Test ID	test-sse-broadcast-performance
Initial State	MessageAnnouncer with multiple listeners registered
Input	Rapid announce() calls with status updates
Expected Results	All listeners receive messages; full queues drop messages silently
Success	Pass

Test ID	test-system-status-caching
Initial State	SystemStatusMonitor with mocked jtop
Input	Multiple getter calls between updates
Expected Results	Cached values returned immediately; no redundant hardware queries
Success	Pass

Test ID	test-watchdog-timeout-accuracy
Initial State	Watchdog with specific timeout value
Input	refresh() calls at various intervals
Expected Results	Timer created with correct timeout; callback triggered after timeout
Success	Pass

5.3 Robustness and Reliability

Unit tests verify error handling, fault tolerance, and thread safety.

Test ID	test-state-error-handling
Initial State	StateManagement with mocked components that may raise exceptions
Input	Various operations with components configured to raise exceptions
Expected Results	Operations fail gracefully; no unhandled exceptions propagate
Success	Pass

Test ID	test-api-error-responses
Initial State	Flask test client
Input	Invalid JSON, missing required fields, malformed requests
Expected Results	HTTP 400 with descriptive error messages; no server crashes
Success	Pass

Test ID	test-sse-thread-safety
Initial State	MessageAnnouncer with concurrent operations
Input	Simultaneous listen(), remove(), and announce() from multiple threads
Expected Results	No race conditions; all operations complete safely
Success	Pass

Test ID	test-ipc-buffer-overflow-handling
Initial State	IPCBufferReceiver with head ahead of tail (overflow condition)
Input	receive() call during overflow
Expected Results	Tail advanced to catch up; data retrieved without corruption
Success	Pass

Test ID	test-cv-management-exception-safety
Initial State	CVProcessManagement with connection in various states
Input	Operations during broken pipe, EOF, and other error conditions
Expected Results	Exceptions caught and swallowed; component continues operating
Success	Pass

Test ID	test-database-error-handling
Initial State	Database with simulated filesystem errors
Input	Operations with mocked os/path functions raising OSError
Expected Results	Graceful degradation; sensible defaults returned; exceptions handled
Success	Pass

Test ID	test-rocam-provider-error-handling
Initial State	RocamProvider with mocked API that fails to connect
Input	API initialization failure
Expected Results	Error state set; error message displayed; component does not crash
Success	Pass

Test ID	test-ui-error-handling
Initial State	Components with mocked API that returns errors
Input	API errors, network failures, malformed responses
Expected Results	Graceful error display; component remains interactive
Success	Pass

Test ID	test-formatters-edge-cases
Initial State	Formatter functions loaded
Input	Edge values (null, undefined, negative, zero, very large numbers)
Expected Results	Sensible defaults; no crashes; formatted output for all inputs
Success	Pass

6 Comparison to Existing Implementation

Not applicable, there is no existing implementation.

7 Unit Testing

7.1 Backend Unit Tests (pytest)

7.1.1 Module: State Management

Test ID	test-state-management-arm-disarm
Initial State	StateManagement instance with mocked dependencies
Input	Call arm() and disarm() methods
Expected Results	Internal armed flag set appropriately; gimbal LED commands issued
Success	Pass

Test ID	test-state-management-manual-move
Initial State	Disarmed system with gimbal at known position
Input	manual_move() with directions up/down/left/right
Expected Results	Gimbal set_deg() called with correctly adjusted angles
Success	Pass

Test ID	test-state-management-manual-move-clamped
Initial State	Disarmed system with gimbal near limits
Input	manual_move() commands that would exceed gimbal limits
Expected Results	Angles clamped to valid ranges (tilt 0–90°, pan -45° to 45°)
Success	Pass

Test ID	test-state-management-recording
Initial State	System with mocked database and CV process
Input	start_recording() and stop_recording() calls
Expected Results	Recording allocated in database; CV process notified; recording ID tracked
Success	Pass

Test ID	test-state-management-download-pause
Initial State	Active recording session
Input	on_download_start() and on_download_end() calls during recording
Expected Results	CV pipeline paused on first download, resumed after last download ends
Success	Pass

Test ID	test-bounding-box-collection
Initial State	Empty BoundingBoxCollection
Input	Sequential CVDData with bounding boxes and timestamps
Expected Results	Correct bbox retrieval within 40ms window; latest valid bbox returned; max 10 entries enforced
Success	Pass

Test ID	test-state-status-performance
Initial State	StateManagement with mocked dependencies and cached metrics
Input	status() called repeatedly
Expected Results	Status returned within milliseconds; cached values returned without recalculation
Success	Pass

Test ID	test-state-error-handling
Initial State	StateManagement with mocked components that may raise exceptions
Input	Various operations with components configured to raise exceptions
Expected Results	Operations fail gracefully; no unhandled exceptions propagate
Success	Pass

7.1.2 Module: API Gateway

Test ID	test-api-endpoints-exist
Initial State	Flask test client with mocked StateManagement
Input	HTTP requests to /api/arm, /api/disarm, /api/manual_move, /api/manual_move_to, /api/recordings/start, /api/recordings/stop
Expected Results	HTTP 200 responses; correct methods called on StateManagement
Success	Pass

Test ID	test-api-recordings-crud
Initial State	Flask test client with mocked database
Input	GET /api/recordings, PATCH /api/recordings/{id}, DELETE /api/recordings/{id}
Expected Results	Recording list returned; rename and delete operations delegated to database
Success	Pass

Test ID	test-api-validation
Initial State	Flask test client
Input	PATCH /api/recordings/{id} with missing or invalid new_name
Expected Results	HTTP 400 Bad Request with appropriate error message
Success	Pass

Test ID	test-api-error-responses
Initial State	Flask test client
Input	Invalid JSON, missing required fields, malformed requests
Expected Results	HTTP 400 with descriptive error messages; no server crashes
Success	Pass

7.1.3 Module: Recording Database

Test ID	test-database-allocate
Initial State	Empty temporary directory
Input	allocate_recording() call
Expected Results	Recording directory created with meta.json; RecordingInfo returned with valid paths
Success	Pass

Test ID	test-database-list-recordings
Initial State	Database with multiple recordings
Input	list_all_recordings() call
Expected Results	Sorted list of RecordingInfo objects (newest first)
Success	Pass

Test ID	test-database-rename-delete
Initial State	Database with existing recording
Input	rename_recording() and delete_recording() calls
Expected Results	meta.json updated with new name; directory removed on delete
Success	Pass

Test ID	test-database-path-traversal-protection
Initial State	Database with validation enabled
Input	Recording IDs containing path traversal patterns (../etc/passwd)
Expected Results	ValueError raised; no filesystem access outside base path
Success	Pass

Test ID	test-database-space-usage
Initial State	Mocked disk usage
Input	space_usage.bytes() and recording_duration_left.s() calls
Expected Results	Correct used/total bytes; estimated recording duration based on free space
Success	Pass

Test ID	test-database-error-handling
Initial State	Database with simulated filesystem errors
Input	Operations with mocked os/path functions raising OSError
Expected Results	Graceful degradation; sensible defaults returned; exceptions handled
Success	Pass

7.1.4 Module: Gimbal Abstraction

Test ID	test-gimbal-crc8
Initial State	GimbalSerial instance
Input	Various byte sequences for CRC calculation
Expected Results	Correct CRC-8 values; consistent results; valid byte range (0–255)
Success	Pass

Test ID	test-gimbal-packet-construction
Initial State	GimbalSerial with mocked serial port
Input	create_request_data() with various request IDs and payloads
Expected Results	Properly formatted packets with CRC at byte 0, request ID at byte 1
Success	Pass

Test ID	test-gimbal-angle-commands
Initial State	GimbalSerial with mocked serial port returning ACK
Input	set_deg(tilt, pan) and get_deg() calls
Expected Results	True on success; correct angle values parsed from response
Success	Pass

Test ID	test-gimbal-led-control
Initial State	GimbalSerial with mocked serial port
Input	set_arm_led(True/False) and set_status_led(True/False)
Expected Results	True on success; serial write called with correct packets
Success	Pass

Test ID	test-gimbal-info-query
Initial State	GimbalSerial with mocked response
Input	gimbal.info() call
Expected Results	Tilt range, pan range, and focal length range parsed correctly
Success	Pass

7.1.5 Module: Tracking

Test ID	test-tracking-queue-behavior
Initial State	Tracking instance with mocked gimbal
Input	Sequential on_detection() calls with bounding box coordinates
Expected Results	Detections queued; oldest dropped when queue full; None accepted
Success	Pass

Test ID	test-tracking-worker-control-law
Initial State	Tracking instance with mocked gimbal at known position
Input	Detection coordinates off-center from frame
Expected Results	gimbal.set_deg() called with corrected angles based on proportional control
Success	Pass

Test ID	test-tracking-angle-clamping
Initial State	Tracking instance with mocked gimbal near angle limits
Input	Detection that would push angles beyond valid range
Expected Results	New angles clamped to valid tilt/pan ranges
Success	Pass

Test ID	test-tracking-stop
Initial State	Running Tracking instance
Input	stop() call
Expected Results	Stop event set; worker thread joined successfully
Success	Pass

7.1.6 Module: CV Process Management

Test ID	test-cv-management-osd-send
Initial State	CVProcessManagement with open IPC connection
Input	send_osd_data() with OSDData
Expected Results	Data sent over IPC; exceptions swallowed safely
Success	Pass

Test ID	test-cv-management-recording-commands
Initial State	CVProcessManagement with open IPC connection
Input	start_recording() with RecordingInfo; stop_recording()
Expected Results	Recording info sent; StopRecording sentinel sent
Success	Pass

Test ID	test-cv-management-pause-resume
Initial State	CVProcessManagement with active process
Input	pause_pipeline() and resume_pipeline() calls
Expected Results	Process terminated on pause; resume event set on resume
Success	Pass

Test ID	test-cv-management-recv-loop
Initial State	CVProcessManagement with mocked connection
Input	Simulated CVData and PreviewData via recv loop
Expected Results	Callbacks dispatched correctly; connection closed on EOF
Success	Pass

Test ID	test-cv-management-exception-safety
Initial State	CVProcessManagement with connection in various states
Input	Operations during broken pipe, EOF, and other error conditions
Expected Results	Exceptions caught and swallowed; component continues operating
Success	Pass

7.1.7 Module: Live Video Process Management

Test ID	test-livestream-construction
Initial State	System ready for process management
Input	LivestreamProcessManagement instantiation
Expected Results	Daemon thread started for process supervision
Success	Pass

Test ID	test-livestream-process-loop
Initial State	LivestreamProcessManagement instance
Input	Simulated process loop iteration
Expected Results	subprocess.Popen called with livestream command; process monitored
Success	Pass

7.1.8 Module: Transcode API

Test ID	test-transcode-routes-404
Initial State	Flask app with transcode routes registered; recording not found
Input	GET /api/recordings/{id}/preview-stabilized and /download-stabilized
Expected Results	HTTP 404 when recording not found
Success	Pass

Test ID	test-transcode-streaming
Initial State	Flask app with valid recording
Input	GET preview-stabilized or download-stabilized with valid recording ID
Expected Results	HTTP 200 with video/webm content type; on_download_start/end called
Success	Pass

Test ID	test-transcode-pipe-cleanup
Initial State	Named pipe exists on filesystem
Input	_cleanup_pipe() call
Expected Results	Pipe file removed; no error if pipe missing; OSError swallowed
Success	Pass

7.1.9 Module: Transcode Process

Test ID	test-transcode-read-log
Initial State	TranscodeProcess instance with mocked GStreamer
Input	JSONL log file with OSD entries
Expected Results	List of OSDData objects parsed correctly; invalid lines skipped
Success	Pass

Test ID	test-transcode-shader-probe
Initial State	TranscodeProcess with OSD data list
Input	Shader probe calls with various PTS values
Expected Results	Correct OSD data selected; pointer advanced appropriately for preview vs download modes
Success	Pass

Test ID	test-transcode-bus-call
Initial State	TranscodeProcess with mocked GStreamer bus
Input	EOS, WARNING, and ERROR messages
Expected Results	Loop quit on EOS and ERROR; continued on WARNING
Success	Pass

7.1.10 Module: CV Process

Test ID	test-cv-format-time
Initial State	CV process utilities loaded
Input	Millisecond timestamps
Expected Results	Human-readable formatted time strings with milliseconds
Success	Pass

Test ID	test-cv-update-osd
Initial State	Mocked GStreamer osd and shader elements
Input	OSDData with gimbal angles, GPS coordinates, IP addresses
Expected Results	Text property set with formatted OSD; shader uniforms updated
Success	Pass

7.1.11 Module: Common IPC

Test ID	test-bounding-box-center
Initial State	BoundingBox instance
Input	Various bounding box coordinates
Expected Results	Correct center point (cx, cy) calculated
Success	Pass

Test ID	test-bounding-box-rotate-90
Initial State	BoundingBox instance
Input	Bounding box for 90-degree rotated frame
Expected Results	Correctly transformed bounding box
Success	Pass

Test ID	test-dataclass-construction
Initial State	IPC module loaded
Input	Construction of CVData, OSDDData, PreviewData, RecordingInfo, StopRecording
Expected Results	Objects created with correct field values
Success	Pass

7.1.12 Module: Common IPC Buffer

Test ID	test-ipc-buffer-sender
Initial State	IPCBufferSender with mocked shared memory
Input	send() calls with fixed-size payloads
Expected Results	Head advanced; data written at correct offset; returns False when full
Success	Pass

Test ID	test-ipc-buffer-receiver
Initial State	IPCBufferReceiver with mocked shared memory containing data
Input	receive() calls with various head/tail positions
Expected Results	Data retrieved correctly; tail advanced; overrun handled gracefully
Success	Pass

Test ID	test-ipc-buffer-overrun-handling
Initial State	IPCBufferReceiver with head significantly ahead of tail (overrun condition)
Input	receive() call during overrun
Expected Results	Tail advanced to catch up; data retrieved without corruption
Success	Pass

Test ID	test-ipc-buffer-memory-safety
Initial State	IPCBuffer with various head/tail positions including edge cases
Input	Operations at buffer boundaries
Expected Results	No out-of-bounds access; proper ring buffer wrapping
Success	Pass

7.1.13 Module: Common Watchdog

Test ID	test-watchdog-refresh
Initial State	Watchdog instance with timeout and callback
Input	refresh() calls
Expected Results	Timer created and started; previous timer cancelled
Success	Pass

Test ID	test-watchdog-clear
Initial State	Watchdog with active timer
Input	clear() call
Expected Results	Timer cancelled; reference cleared; idempotent when no timer
Success	Pass

Test ID	test-watchdog-timeout-accuracy
Initial State	Watchdog with specific timeout value
Input	refresh() calls at various intervals
Expected Results	Timer created with correct timeout; callback triggered after timeout
Success	Pass

7.1.14 Module: System Status

Test ID	test-system-status-caching
Initial State	SystemStatusMonitor with mocked jtop
Input	Multiple getter calls between updates
Expected Results	Cached values returned immediately; no redundant hardware queries
Success	Pass

Test ID	test-system-status-callback-updates
Initial State	SystemStatusMonitor with initial readings
Input	jtop callback with updated sensor values
Expected Results	All cached metrics updated to new values
Success	Pass

7.1.15 Module: Common Utils

Test ID	test-ip4-addresses
Initial State	Utils module with mocked netifaces
Input	Network interface configurations with various IPv4 addresses
Expected Results	List of IP addresses extracted correctly; empty list when no interfaces
Success	Pass

Test ID	test-scheduler-validation
Initial State	Scheduler functions with mocked OS calls
Input	Various priority values (valid and invalid) for FIFO, OTHER, and BATCH schedulers
Expected Results	ValueError raised for out-of-range priorities; successful calls for valid values
Success	Pass

7.1.16 Module: Recording-Only State Management

Test ID	test-state-management-recording-only-noops
Initial State	StateManagementRecordingManagementOnly instance
Input	Calls to arm(), disarm(), manual_move(), manual_move_to(), start_recording(), stop_recording()
Expected Results	All operations are no-ops (do not raise exceptions)
Success	Pass

Test ID	test-state-management-recording-only-status
Initial State	StateManagementRecordingManagementOnly with mocked database and system status
Input	status() call
Expected Results	StatusResponse with is_recording=False, armed=False, system metrics populated
Success	Pass

Test ID	test-control-process-main-entry
Initial State	Mocked StateManagement and threading
Input	run_control_process() and run_recording_management() calls
Expected Results	API thread started and joined; StateManagement instantiated
Success	Pass

7.1.17 Module: Main Entry Point

Test ID	test-main-dispatch-control
Initial State	Mocked sys.argv with no arguments
Input	Default execution (no command line args)
Expected Results	run_control_process() called
Success	Pass

Test ID	test-main-dispatch-cv
Initial State	Mocked sys.argv with ["cv"] argument
Input	CV process launch command
Expected Results	run_cv_process() called
Success	Pass

Test ID	test-main-dispatch-livestream
Initial State	Mocked sys.argv with ["livestream"] argument
Input	Livestream process launch command
Expected Results	run_livestream_process() called
Success	Pass

Test ID	test-main-dispatch-transcode
Initial State	Mocked sys.argv with ["download-stabilized"] or ["preview-stabilized"] arguments
Input	Transcode process launch command with video and log paths
Expected Results	start_transcode_process() called with correct mode and paths
Success	Pass

Test ID	test-main-dispatch-unknown
Initial State	Mocked sys.argv with unknown command
Input	Unknown command line argument
Expected Results	System exits with code 1
Success	Pass

7.1.18 Module: SSE

Test ID	test-sse-broadcast-performance
Initial State	MessageAnnouncer with multiple listeners registered
Input	Rapid announce() calls with status updates
Expected Results	All listeners receive messages; full queues drop messages silently (no blocking)
Success	Pass

Test ID	test-sse-thread-safety
Initial State	MessageAnnouncer with concurrent operations
Input	Simultaneous listen(), remove(), and announce() from multiple threads
Expected Results	No race conditions; all operations complete safely
Success	Pass

7.2 Frontend Unit Tests (Vitest)

Test ID	test-control-page-render
Initial State	React component with mocked dependencies
Input	Various status states (null, with/without preview, armed/disarmed, recording)
Expected Results	Preview image rendered when available; spinner shown when loading; REC/ARMED indicators displayed correctly
Success	Pass

Test ID	test-control-page-bbox-overlay
Initial State	Control page with status containing bbox
Input	Status with bounding box coordinates and confidence
Expected Results	Bounding box overlay rendered with confidence value
Success	Pass

Test ID	test-controls-card-render
Initial State	ControlsCard component with mocked API
Input	Various system states (armed/disarmed)
Expected Results	Directional controls rendered; manual move API called on interaction
Success	Pass

Test ID	test-control-page-drag
Initial State	Control page in disarmed mode
Input	Pointer drag gestures on preview area
Expected Results	manualMoveTo API called with calculated tilt/pan values
Success	Pass

Test ID	test-system-status-card
Initial State	SystemStatusCard component with status payload fixtures
Input	Status payload fixtures with CPU/FPS/temperature/GPS combinations
Expected Results	Correct metric text rendering and safe handling when status is null
Success	Pass

Test ID	test-navbar
Initial State	Rendered navbar with mocked fullscreen API
Input	Navbar interactions (link clicks, configuration toggle, fullscreen)
Expected Results	Required navigation links exist; fullscreen requests/exit call correct browser API
Success	Pass

Test ID	test-recordings-page-loading
Initial State	Recordings page mounted
Input	API loading state
Expected Results	Loading spinner displayed
Success	Pass

Test ID	test-recordings-page-empty
Initial State	Recordings page with empty list
Input	API returns empty recordings array
Expected Results	“No recordings” message displayed
Success	Pass

Test ID	test-recordings-page-list
Initial State	Recordings page with sample data
Input	API returns list of recordings
Expected Results	Recording names rendered; dates and durations formatted
Success	Pass

Test ID	test-recordings-delete
Initial State	Recordings page with sample recording
Input	Delete button clicked with confirmation
Expected Results	deleteRecording API called with correct ID
Success	Pass

Test ID	test-recordings-rename
Initial State	Recordings page with editable name
Input	Name changed and blurred (or Escape pressed)
Expected Results	renameRecording API called with new name; cancelled on Escape
Success	Pass

Test ID	test-language-selector
Initial State	LanguageSelector component
Input	Language selection change
Expected Results	Language atom updated; i18n activated
Success	Pass

Test ID	test-configuration-menu
Initial State	ConfigurationMenu component
Input	Settings changes (temperature unit, drag sensitivity, invert drag)
Expected Results	Setting atoms updated with new values
Success	Pass

Test ID	test-api-client-urls
Initial State	ApiClient instance
Input	Base URL configuration
Expected Results	Correct endpoint URLs generated for status stream, recordings, previews
Success	Pass

Test ID	test-api-client-automatic-discovery
Initial State	Network with multiple potential server URLs
Input	createAutomatic() with probe URLs
Expected Results	Client created with first responding URL; throws if all fail
Success	Pass

Test ID	test-api-client-methods
Initial State	ApiClient with mocked fetch
Input	manualMove(), arm(), disarm(), startRecording(), stopRecording(), listRecordings(), renameRecording(), deleteRecording()
Expected Results	Correct HTTP methods and URLs; proper request bodies; ApiError thrown on failure
Success	Pass

Test ID	test-rocam-provider-initialization
Initial State	RocamProvider component with mocked createAutomatic
Input	Provider mounted with children
Expected Results	API client created; status polling started; loading state managed
Success	Pass

Test ID	test-rocam-provider-error-handling
Initial State	RocamProvider with mocked API that fails to connect
Input	API initialization failure
Expected Results	Error state set; error message displayed; component doesn't crash
Success	Pass

Test ID	test-rocam-provider-polling
Initial State	RocamProvider with mocked API client
Input	Status stream events and polling intervals
Expected Results	Status state updated; polling continues with proper cleanup on unmount
Success	Pass

Test ID	test-use-rocam-hook
Initial State	Component wrapped in RocamProvider
Input	useRocam() hook called
Expected Results	API client and status accessible to consuming components
Success	Pass

Test ID	test-formatters-degrees
Initial State	Formatter functions loaded
Input	Various numeric values for degrees, FPS, percentages
Expected Results	Correctly formatted strings with appropriate precision and units
Success	Pass

Test ID	test-formatters-temperature
Initial State	Formatter functions loaded
Input	Celsius temperatures; Fahrenheit unit preference
Expected Results	Correct conversion and formatting with degree symbol
Success	Pass

Test ID	test-formatters-bytes
Initial State	Formatter functions loaded
Input	Byte counts of various magnitudes
Expected Results	Human-readable sizes (B, KB, MB, GB)
Success	Pass

Test ID	test-formatters-duration
Initial State	Formatter functions loaded
Input	Millisecond durations
Expected Results	Formatted HH:MM:SS or relative time strings
Success	Pass

Test ID	test-i18n-activation
Initial State	i18n module with mocked linguist
Input	dynamicActivate() with locale string
Expected Results	Messages loaded; i18n activated with correct locale
Success	Pass

Test ID	test-app-routing
Initial State	App component with MemoryRouter
Input	Navigation to / and /recordings routes
Expected Results	Correct page components rendered; unknown paths redirect to /
Success	Pass

Test ID	test-formatters-edge-cases
Initial State	Formatter functions loaded
Input	Edge values (null, undefined, negative, zero, very large numbers)
Expected Results	Sensible defaults; no crashes; formatted output for all inputs
Success	Pass

Test ID	test-formatters-internationalization
Initial State	Formatters with various locale considerations
Input	Values requiring locale-specific formatting
Expected Results	Consistent formatting regardless of system locale
Success	Pass

Test ID	test-ui-loading-states
Initial State	Components with loading/null states
Input	Null or undefined data
Expected Results	Loading spinners or placeholder content; no crashes
Success	Pass

Test ID	test-ui-error-handling
Initial State	Components with mocked API that returns errors
Input	API errors, network failures, malformed responses
Expected Results	Graceful error display; component remains interactive
Success	Pass

8 Changes Due to Testing

The V&V process directly changed implementation details in both the frontend and backend. The following tables summarize the verification evidence, the resulting changes, and their impact on project quality.

Table 2: Usability and Safety Changes from V&V

V&V Evidence	Change, and Impact on Quality
Stakeholder walkthrough feedback + test-control-page-drag (pass)	Added drag-to-move gimbal control on the preview area, in addition to the directional buttons. This improved usability and operator efficiency by enabling more direct camera framing and faster manual setup.
test-confirmation-prompts (pass) and safety walkthrough observations	Added and standardized blocking confirmation prompts for hazardous actions such as force home, hard stop, high-speed slew, and reset/update paths. This reduced the risk of accidental unsafe operations and made safety-critical actions more explicit and deliberate.

Table 3: UI and Recording Changes from V&V

V&V Evidence	Change, and Impact on Quality
test-visual-indicators (pass), test-report-errors (pass), and test-ui-error-handling (pass)	Improved UI status and error messaging for armed, tracking, target-lost, network, storage, and device-error states; ensured failures are visible to the operator. This increased operational awareness and robustness by making failures visible, actionable, and non-crashing.
test-state-management-recording (pass), test-state-management-downloading (pass), and test-manage-recordings (pass)	Refined recording-state behaviour so downloads pause and resume correctly and the recording workflow remains stable during use. This improved reliability of recording management and reduced the risk of recording interruption or data loss during operator interaction.

Table 4: Backend Validation Changes from V&V

V&V Evidence	Change, and Impact on Quality
test-api-validation (pass), test-validate-commands (pass), and test-api-error-responses (pass)	Strengthened backend request and command validation, with consistent HTTP 400 responses for malformed or invalid inputs. This improved robustness and safety by rejecting invalid commands before they can propagate into control paths.

Overall, these updates show that verification results were used to make concrete product improvements, not only to report pass/fail status.

8.1 Planned Future Improvements

The following improvements have been identified through the V&V process and stakeholder feedback but have **not yet been implemented** due to time and resource constraints. They are documented here for future development iterations.

Table 5: Planned Future Improvements Identified During V&V

Source of Feedback	Planned Change
Peer Review (#342)	Conduct formal usability testing with end-users (task completion timing, satisfaction surveys, SUS scoring) and summarize results with charts and statistical analysis.
Stakeholder Review	Perform field validation at an actual rocket launch site to confirm tracking performance, latency, and HDMI output quality under real flight conditions (wind, distance, lighting variability).
TA Feedback	Extend code coverage targets to 90% line coverage for safety-critical modules (state management, gimbal abstraction) and add mutation testing for critical state transitions.
Testing Observations	Add automated regression tests for the HDMI output pipeline and recording workflow to detect freeze or corruption issues without manual intervention.

9 Automated Testing

Unit tests are automated by GitHub Actions. On every pull request targeting `main`, the backend CI workflow runs linting and tests for changes under `src/backend/`, and the frontend CI workflow runs linting and tests for changes under `src/react-app/`. All tests must pass before a pull request can be merged.

10 Trace to Requirements

Test ID	Requirements
test-acquire-frame	FR-1, SLR-2
test-disconnect-camera	FR-1
test-hdmi-output	FR-8, FR-9, INT-1, SLR-3
test-disconnect-hdmi	FR-8
test-preview-real-time	FR-10
test-network-disconnect	FR-10
test-recording	FR-11
test-disk-full	FR-11
test-manage-recordings	FR-12
test-manual-control	FR-2, FR-6
test-transition-to-tracking	FR-3, FR-4
test-tracking	FR-2, FR-3, FR-7
test-return-to-idle	FR-5
test-exit-gimbal-range	FR-2, FR-7
test-consistent-units	EZ-1, PI-1
test-bilingual-ui	PI-2
test-immediate-ui-feedback	EZ-2
test-visual-indicators	EZ-3
test-confirmation-prompts	EZ-4, UPR-3
test-manual-idle-mode	SCR-1
test-no-collection-pii	LR-1
test-color-vision-deficiency	AR-1
test-manual-arm-mode	SCR-4
test-gimbal-interface	INT-2
test-report-errors	RFR-1
test-stop-immediately	RFR-2
test-validate-commands	IR-1
test-log-operations	AUR-1

11 Trace to Modules

Table 6: Traceability Between Unit Test Cases and Backend Modules

Module	Unit Test Cases
State Management	test-state-management-arm-disarm; test-state-management-manual-move; test-state-management-manual-move-clamped; test-state-management-recording; test-state-management-download-pause; test-state-status-performance; test-state-error-handling
API Gateway	test-api-endpoints-exist; test-api-recordings-crud; test-api-validation; test-api-error-responses
Recording Database	test-database-allocate; test-database-list-recordings; test-database-rename-delete; test-database-path-traversal-protection; test-database-space-usage; test-database-error-handling
Gimbal Abstraction	test-gimbal-crc8; test-gimbal-packet-construction; test-gimbal-angle-commands; test-gimbal-led-control; test-gimbal-info-query
Tracking	test-tracking-queue-behavior; test-tracking-worker-control-law; test-tracking-angle-clamping; test-tracking-stop
CV Process Management	test-cv-management-osd-send; test-cv-management-recording-commands; test-cv-management-pause-resume; test-cv-management-recv-loop; test-cv-management-exception-safety
Live Video Management	test-livestream-construction; test-livestream-process-loop
Transcode Management	test-transcode-read-log; test-transcode-shader-probe; test-transcode-bus-call
Transcode Process	test-transcode-read-log; test-transcode-shader-probe; test-transcode-bus-call
CV Process	test-cv-format-time; test-cv-update-osd
Live Video Process	test-livestream-construction; test-livestream-process-loop
Common IPC	test-bounding-box-center; test-bounding-box-rotate-90; test-dataclass-construction
Common IPC Buffer	test-ipc-buffer-sender; test-ipc-buffer-receiver; test-ipc-buffer-overflow-handling; test-ipc-buffer-memory-safety

Continued on next page

Table 6 – Continued from previous page

Module	Unit Test Cases
Common Watchdog	<code>test-watchdog-refresh;</code> <code>test-watchdog-clear;</code> <code>test-watchdog-timeout-accuracy</code>
Common System Status	<code>test-system-status-caching;</code> <code>test-system-status-callback-updates</code>
Common Utils	<code>test-ip4-addresses;</code> <code>test-scheduler-validation</code>
Recording-Only State Management	<code>test-state-management-recording-only-noops;</code> <code>test-state-management-recording-only-status;</code> <code>test-control-process-main-entry</code>
Main Entry Point	<code>test-main-dispatch-control;</code> <code>test-main-dispatch-cv;</code> <code>test-main-dispatch-livestream;</code> <code>test-main-dispatch-transcode;</code> <code>test-main-dispatch-unknown</code>
Transcode API	<code>test-transcode-routes-404;</code> <code>test-transcode-streaming;</code> <code>test-transcode-pipe-cleanup</code>
SSE	<code>test-sse-broadcast-performance;</code> <code>test-sse-thread-safety</code>

Table 7: Traceability Between Unit Test Cases and Frontend Modules

Module	Unit Test Cases
Preview	<code>test-control-page-render;</code> <code>test-control-page-bbox-overlay</code>
Manual Control	<code>test-controls-card-render;</code> <code>test-control-page-drag</code>
Recording Management	<code>test-recordings-page-loading;</code> <code>test-recordings-page-empty;</code> <code>test-recordings-page-list;</code> <code>test-recordings-delete;</code> <code>test-recordings-rename</code>
Configuration	<code>test-language-selector;</code> <code>test-configuration-menu</code>
API Client	<code>test-api-client-urls;</code> <code>test-api-client-automatic-discovery;</code> <code>test-api-client-methods</code>
Rocam Provider	<code>test-rocam-provider-initialization;</code> <code>test-rocam-provider-error-handling;</code> <code>test-rocam-provider-polling;</code> <code>test-use-rocam-hook</code>
Utility Formatters	<code>test-formatters-degrees;</code> <code>test-formatters-temperature;</code> <code>test-formatters-bytes;</code> <code>test-formatters-duration;</code> <code>test-formatters-edge-cases;</code> <code>test-formatters-internationalization</code>
Internationalization	<code>test-i18n-activation</code>
App (Routing)	<code>test-app-routing</code>
UI Components (Individual)	<code>test-system-status-card;</code> <code>test-navbar;</code> <code>test-controls-card;</code> <code>test-language-selector;</code> <code>test-configuration-menu;</code> <code>test-ui-loading-states;</code> <code>test-ui-error-handling</code>

12 Code Coverage Metrics

Unit tests were executed with coverage instrumentation to measure the proportion of source code exercised by the test suite. Table 8 summarizes coverage for both the backend (Python, pytest-cov) and frontend (React/TypeScript, Vitest v8).

Table 8: Code Coverage Metrics

Metric	Backend	Frontend
Test cases passed	313	168
Line coverage	88%	85.57%
Statement coverage	88%	82.41%
Branch coverage	81.4%	70.35%
Function coverage	—	85.52%
Statements (total)	1,349	—
Statements covered	1,198	—

References

- Vision-Guided Tracker Team. Software architecture (module guide), 2025a. Project document, this repository.
- Vision-Guided Tracker Team. Software detailed design (module interface specification), 2025b. Project document, this repository.
- Vision-Guided Tracker Team. Software requirements specification, 2025c. Project document, this repository.
- Vision-Guided Tracker Team. Verification and validation plan, 2025d. Project document, this repository.

Appendix — Reflection

The information in this section will be used to evaluate the team members on the graduate attribute of Reflection.

1. What went well while writing this deliverable?

Writing this deliverable went well because our team worked smoothly and had a clear division of work. Different members were responsible for different parts of the testing effort, especially backend and frontend unit tests, so people could work in parallel instead of waiting on each other. Some team members focused more on backend areas such as state management, API validation, recording database behaviour, gimbal communication, and tracking logic, while others focused more on frontend areas such as the control page, recordings page, UI feedback, configuration menu, provider behaviour, and formatting logic. Because the work was split clearly, progress was steady and integration was faster.

Another thing that went well was that our system already had a modular structure, so it was easier to organize the testing and later write the report. Since the project already had clear pieces such as backend control logic, frontend React UI, tracking logic, CV process management, recording management, and transcode-related components, it was easier to connect each test to a module and then connect those modules back to requirements. This helped a lot when writing the traceability sections of the VnV Report because we were not trying to force tests into an unclear structure at the last minute.

The iteration speed also helped a lot. As tests were added and updated, we could quickly see whether backend and frontend changes were still working. For example, once tests for API validation, recording management, provider polling, and control-page interactions were in place, it was much easier to organize the evidence into the document. Overall, the writing process felt smooth because implementation, testing, and documentation were moving together instead of being done completely separately at the end.

2. What pain points did you experience during this deliverable, and how did you resolve them?

One pain point was deciding how to present the testing methods clearly. Our project is not just a normal software application. It includes frontend code, backend code, real-time status updates, tracking logic, gimbal control, recording workflows, and hardware-related behaviour. Because of this, it was sometimes difficult to decide what should be described as a unit test, what should be treated as a system test, and what should be explained as a walkthrough or manual verification activity. For example, API validation tests, provider tests, and transcode route tests are automated, but they are closer to component-level behaviour than very small isolated units. We resolved this by focusing more on whether each test gave useful verification evidence instead of worrying too much about the exact label.

Another pain point was coverage and test metrics. It was easy to report line coverage and the number of passing tests, but it was harder to explain what those numbers really mean for our system. For example, branch coverage shows whether different decision paths were exercised, e.g., success versus failure cases, valid versus invalid API input, empty versus non-empty recording lists, or safe versus unsafe state transitions. However, even strong branch coverage does not fully prove that the full system will behave well in real launch-like conditions. Some of the main risks in our project are in hardware interaction, timing, and real-time behaviour, and those are not fully captured by normal coverage numbers. We resolved this by treating coverage as one useful metric, e.g., line coverage, branch coverage, and passing test counts, but not the only proof of quality, and by clearly stating that field testing was out of scope for this iteration.

A third pain point was that our system depends heavily on hardware and integration. Some behaviours, such as actual gimbal response, live camera input stability, real tracking performance, and end-to-end timing under realistic outdoor conditions, are much harder to verify than pure software logic. These parts cannot always be tested in a simple automated way using mocks. For example, we can test gimbal angle clamping logic and invalid command rejection in software, but that is still different from validating how the full gimbal responds under real motion and camera conditions. We resolved this by putting more effort into the parts we could verify well in the current course scope, such as state transitions, API validation, recording management, transcode logic, UI feedback, and safety-related interface behaviour, and then being honest in the report about what was not executed.

Another smaller pain point was avoiding repetition. Many tests supported more than one purpose at the same time. A single test could support module-level verification, nonfunctional discussion, and traceability to requirements all at once. For example, a test like API error handling can support both robustness discussion and backend module traceability, while a test like control-page drag can support both usability and frontend module verification. We resolved this by using traceability tables and references between sections so the same evidence could be reused without rewriting the same explanation too many times.

3. Which parts of this document stemmed from speaking to your client(s) or a proxy (e.g. your peers)? Which ones were not, and why?

The parts of this document that were most influenced by speaking to our client or proxy were the parts related to usability, operator workflow, and changes made after testing. Since the system is meant to be used by an operator during tracking and recording, feedback about how the interface feels in use is very important. In particular, the change that added drag control on the preview area came directly from customer feedback after they tried the product. They wanted a more direct and intuitive way to move the gimbal instead of relying only on directional buttons. That change is reflected both in the implementation and in the section on changes due to testing.

Other usability-related parts of the report were also influenced by user or proxy feedback. For example, tests about immediate UI feedback, visual indicators, manual idle

control, confirmation prompts, bilingual behaviour, and consistent units are closely connected to how a real user experiences the system. The importance of clear operator state information such as armed, tracking, target lost, storage alerts, and network errors also reflects actual usage concerns rather than just internal implementation details.

The parts of the report that were less directly based on client feedback were mostly the deeper engineering and software quality sections. For example, tests related to SSE thread safety, IPC buffer behaviour, API error responses, path traversal protection in the recording database, watchdog timing, formatter edge cases, and transcode process behaviour were mainly driven by software engineering concerns. These were included because they are important for correctness, robustness, maintainability, and safety, even if the client would not explicitly ask for them by name. The same is true for code coverage metrics and some of the traceability work. For example, the client may care that recording works correctly, but they would not usually ask specifically for branch coverage on rename and delete paths or for thread-safety checks on SSE listeners. These parts exist because they are important for demonstrating technical quality and satisfying course expectations, not because they came directly from stakeholder requests.

In short, the user-facing and workflow-related parts of the report were influenced more by the client or proxy, while the lower-level software quality evidence came more from the team's own engineering judgment and the structure required for the course deliverable.

4. **In what ways was the Verification and Validation (VnV) Plan different from the activities that were actually conducted for VnV? If there were differences, what changes required the modification in the plan? Why did these changes occur? Would you be able to anticipate these changes in future projects? If there weren't any differences, how was your team able to clearly predict a feasible amount of effort and the right tasks needed to build the evidence that demonstrates the required quality? (It is expected that most teams will have had to deviate from their original VnV Plan.)**

The actual V&V activities were similar to the original plan in overall goal, but different in scope and in the type of evidence that was most practical to collect. The main goal stayed the same: we wanted to show that the core functions, user interface, recording workflow, and key quality attributes were working correctly. However, in practice, the completed work focused more on automated tests, system tests, walkthroughs, and document review than on broader field-style validation.

The biggest difference was that field testing was not completed in this iteration. That part was especially difficult because it depends on hardware availability, real-time camera setup, gimbal behaviour, environmental conditions, and launch-like scenarios that are not easy to recreate during a course schedule. That means some planned verification activities were harder to execute in a reliable and repeatable way within the time available. As a result, we relied more heavily on evidence that could be collected in a repeatable development setting, such as backend automated tests, frontend automated tests, API tests, mocked component tests, recording workflow tests, and manual

walkthroughs.

Another difference was that some qualities that would ideally be measured through full system or field behaviour had to be supported using partial evidence instead. For example, performance-related claims were supported through tests such as status caching, SSE broadcast behaviour, watchdog timing, and responsiveness of UI actions, rather than through a full outdoor end-to-end latency study. Similarly, robustness and reliability were supported through tests for error handling, invalid requests, concurrency safety, IPC buffer overrun handling, and exception safety, rather than through long-duration field deployment. In other words, we could still verify many important paths, e.g., valid versus invalid commands, connected versus disconnected states, recording start versus stop paths, and normal versus error responses, even if we could not execute every realistic outdoor scenario.

These changes happened mainly because a large part of the project risk is in integration between modules, hardware devices, and real-world operating conditions. That makes V&V planning more difficult, because the team may know what should ideally be tested, but not all of those tests are equally feasible within one iteration. In future projects, we could anticipate this better by separating the plan into clearer categories from the beginning, such as software-only automated tests, local integration tests, manual walkthrough tests, and hardware or field validation planned for later stages. Doing that would make the plan more realistic and would also make the final report match the original plan more closely.

Even though the exact execution differed from the broader original idea, the final V&V work still supported the main quality goals of the project. We were able to provide meaningful evidence for functional correctness, usability, performance-related behaviour, and robustness/reliability, while also documenting the limits of the current scope. Because of that, the final report still reflects the intent of the VnV Plan, even though some planned activities had to be reduced or replaced by more practical forms of verification evidence.